SPKF iteration code, load model

- Implementation of SPKF on nonlinear model refactors code.
  - "Wrapper" code, coordinates the entire simulation process.
  - Initialization routine (`initSPKF.m`), called once at startup.
  - Update routine (`iterSPKF.m`), called every sample interval.
- Code begins by loading model parameters into local variables.

```
function [xhat,bounds,spkfData] = iterSPKF(uk,zk,dT,spkfData)
  model = spkfData.model;
  SQRT = @(x) sqrt(max(0,x)); % "Safe" square root

  % Get data stored in spkfData structure
  priorU = spkfData.priorU;
  SigmaX = spkfData.SigmaX;
  xhat   = spkfData.xhat;
  nx     = spkfData.nx;        nw = spkfData.nw;
  nv     = spkfData.nv;        na = spkfData.na;
  Snoise = spkfData.Snoise;
  Wc     = spkfData.Wc;
```


SPKF iteration code, step 1a(1)

- SPKF iteration code continues:
  - Compute augmented $\hat{x}^{a,+}_{k-1}$, $\sqrt{\Sigma^{a,+}_{\tilde{x},k-1}}$.

```
  % Step 1a: State estimate time update
  %          - Create xhatminus augmented SigmaX points
  %          - Extract xhatminus state SigmaX points
  %          - Compute weighted average xhatminus(k)

  % Step 1a-1: Create augmented SigmaX and xhat
  [sigmaXa,p] = chol(SigmaX,'lower');
  if p>0
    fprintf('Cholesky error.  Recovering...\n');
    theAbsDiag = abs(diag(SigmaX));
    sigmaXa = diag(max(SQRT(theAbsDiag),SQRT(spkfData.Sw/dT)));
    % Alternate implementation with matrix SigmaW
    % sigmaXa = diag(max(SQRT(theAbsDiag),SQRT(spkfData.SigmaW)));
  end
  sigmaXa=[real(sigmaXa) zeros([nx nw+nv]); zeros([nw+nv nx]) Snoise];
  xhata = [xhat; zeros([nw+nv 1])];
  % NOTE: sigmaXa is lower-triangular
```


SPKF iteration code, steps 1a–1b

- SPKF iteration code continues:
  - Compute $\hat{x}^{-}_{k}$ and $\Sigma^{-}_{\tilde{x},k}$.

```
  % Step 1a-2: Calculate SigmaX points (strange indexing of xhata to
  % avoid "repmat" call, which is very inefficient in MATLAB)
  Xa = xhata(:,ones([1 2*na+1])) + ...
       spkfData.h*[zeros([na 1]), sigmaXa, -sigmaXa];

  % Step 1a-3: Time update from last iteration until now
  %     stateEqn(xold,current,xnoise)
  Xx = stateEqn(Xa(1:nx,:),priorU,dT,Xa(nx+1:nx+nw,:));
  xhat = Xx*spkfData.Wm;

  % Step 1b: Error covariance time update
  %          - Compute weighted covariance sigmaminus(k)
  %            (strange indexing of xhat to avoid "repmat" call)
  Xs = Xx - xhat(:,ones([1 2*na+1]));
  SigmaX = Xs*diag(Wc)*Xs';
```

SPKF iteration code, steps 1c–2b

- SPKF iteration code continues:
  - Compute $\hat{y}_k$, L_k, and state estimate $\hat{x}_k^+$.

```
% Step 1c: Output estimate
%          - Compute weighted output estimate yhat(k)
Z = outputEqn(Xx,Xa(nx+nw+1:end,:),model);
zhat = Z*spkfData.Wm;

% Step 2a: Estimator gain matrix
Zs = Z - zhat(:,ones([1 2*na+1]));
SigmaXZ = Xs*diag(Wc)*Zs';
SigmaZ  = Zs*diag(Wc)*Zs';
L = SigmaXZ/SigmaZ;

% Step 2b: State estimate measurement update
r = zk - zhat; % innovation: use to check for sensor errors...
if r^2 > 100*SigmaZ, L(:)=0.0; end
xhat = xhat + L*r;
```


SPKF iteration code, step 2c

- SPKF iteration code continues:
  - Compute $\Sigma_{\tilde{x},k}^+$; adjust if needed; store data for next iteration.

```
% Step 2c: Error covariance measurement update
SigmaX = SigmaX - L*SigmaZ*L';

% Q-bump code
if r^2 > 16*SigmaZ % bad output estimate by 4 std. devs, bump Q
  fprintf('Bumping SigmaX\n');
  SigmaX = SigmaX*spkfData.Qbump;
end
[~,S,V] = svd(SigmaX); % Implement Higham method to help robustness
HH = V*S*V';
SigmaX = (SigmaX + SigmaX' + HH + HH')/4;

% Save data in spkfData structure for next time...
spkfData.priorU = uk;
spkfData.SigmaX = SigmaX;
spkfData.xhat   = xhat;
bounds          = 3*sqrt(diag(SigmaX));
```


Nested state- and output-equation functions

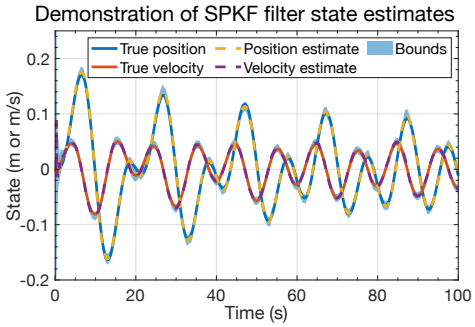
- SPKF functions to implement state, measurement equations:
  - Vector operations across all sigma points simultaneously.

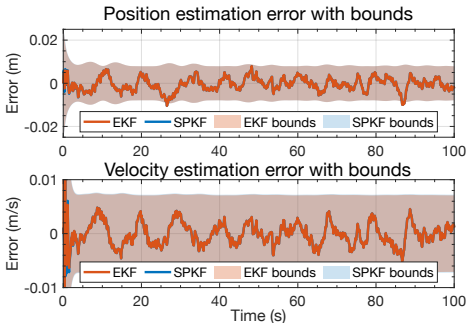
```
% Calculate new states for all of the old state vectors in xold.
function xnew = stateEqn(xold,priorU,dT,xnoise)
  % Compute linear homogeneous part
  xnew = [1 dT; -model.k1*dT/model.m 1-model.b*dT/model.m]*xold;
  % Add on nonlinear and forcing part
  xnew(2,:) = xnew(2,:) - model.k2*dT*xold(1,:).^3/model.m + ...
              dT/model.m*(priorU + xnoise);
  % Alternate implementation with matrix SigmaW ...
  % xnew(2,:) = xnew(2,:) - model.k2*dT*xold(1,:).^3/model.m + ...
  %             dT/model.m*priorU;
  % xnew = xnew + xnoise;
end

% Calculate outputs for all state vectors in xhat
function zhat = outputEqn(xhat,znoise,model)
  zhat = atan(xhat(1,:)/model.d) + znoise;
end
end
```

Example SPKF on nonlinear model results

- SPKF was executed for the dataset presented in Lesson 3.1.5.
  - RMS state-estimation error = 0.0033 (vs. 0.0028 for EKF).
  - Percent of time error outside bounds = 0.45% (same as EKF).
  - EKF/SPKF estimates, bounds are nearly indistinguishable after startup transient.





Summary

- Implementation of SPKF on nonlinear model refactors code.
  - Initialization routine (`initSPKF.m`), called once at startup.
  - Update routine (`iterSPKF.m`), called every sample interval.
  - "Wrapper" code, coordinates the entire simulation process.
- You have now seen the details of the entire codeset plus some example results.
- SPKF works quite well as state estimator using this nonlinear model.